Distributed Data Fundamentals

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

SESSION 1 OF 4

The Big Picture — Why This Session Exists

Your training datasets keep growing. At some point a single machine's disk, RAM, or patience simply runs out. This session is about what happens next.

THINK OF IT LIKE THIS: Moving a small kitchen recipe to feeding a stadium isn't just 'more of the same ingredients.' You need multiple kitchens, a plan for who cooks what, and a way to combine the results. Distributed data processing is that plan for datasets too big for one machine.

1 Data outgrows one machine

Modern datasets are TBs to PBs — routinely bigger than any single disk or RAM.

2 Splitting data is the hard part

How you split it (partition) determines whether later steps are fast or painfully slow.

3 Moving data between machines is expensive

The 'shuffle' — regrouping data across workers — is the most expensive operation in the whole pipeline.

Key Terms Before We Start

Partition — One chunk of a dataset, small enough for one worker to handle.

Worker — One process (usually on one machine) doing a slice of the overall work.

DAG (task graph) — A diagram of steps and their dependencies — what must happen before what.

Lazy evaluation — Building a plan first, only actually running it when the result is needed.

Shuffle — Moving data between workers to regroup it by a key — the expensive operation.

Lineage — The record of how a piece of data was derived, used to recompute it if it's lost.

Why Distributed Data Matters for ML

1 Pretraining corpora

Text pretraining at 1-15 TB compressed — doesn't fit on any single machine's disk, let alone RAM.

2 Image/video datasets

LAION-5B is 240 TB. Must stream from object storage or a parallel filesystem.

3 Batch inference

Scoring 10B rows nightly: one machine takes a week, a cluster does it in hours.

4 Feature engineering

Group-bys and joins over billions of events — Spark territory.

Partitioning — the Single Most Important Choice

A dataset = N partitions, one or more per worker. Partition choice determines everything downstream.

By ROW RANGE

Simple, balanced sizes.
Joins/groupbys trigger a full shuffle.

By HASH(key)

hash(key) picks the worker.
Joins on that key need NO shuffle.

By RANGE(key)

Ordered scans, range queries.
Imbalanced if the key is skewed.

THINK OF IT LIKE THIS: Sorting mail by ZIP code (range) works great if requests come in by ZIP. Sorting by hash of the recipient's name spreads load evenly but loses any ordering.

RULE OF THUMB: partition by the column you'll GROUP or JOIN on most. Then check for skew.

DAGs and Lazy Evaluation

Your code builds a PLAN. The engine optimises and executes only when you ask for results.

```
# PANDAS (EAGER) — every line runs immediately
df = pd.read_csv('events.csv'); df =
df[df.type=='click']; result = df.groupby('hour').count()

# DASK/SPARK (LAZY) — nothing runs until .compute()
df = dd.read_csv('events.csv'); df = df[df.type=='click']
result = df.groupby('hour').count(); result.compute()   #
NOW it runs
```

- Predicate pushdown — filter rows at the disk/format level (Parquet, ORC)
- Projection pushdown — only read columns you'll actually use
- Parallel execution — split the plan across workers automatically

The Shuffle — the Expensive Operation

A shuffle moves data between workers to group it by key. It is BY FAR the most expensive operation in a distributed pipeline.

What triggers one

GROUP BY a non-partition column.
JOIN on a non-partition column.
Global sort. Repartitioning.

Why it's costly

Serialise every row.
Network transfer between N workers.
Spill to disk if memory is tight.

HOW TO MINIMISE: pre-partition on the keys you'll group/join. Broadcast small tables (<100MB) instead of shuffling both sides. Filter before you group.

Fault Tolerance via Lineage

Lost a partition? Re-derive it from its parents. No checkpoints needed, usually.

THINK OF IT LIKE THIS: If you lose your notes for step 5 of a recipe, but you know steps 1-4 exactly, you can just redo step 5 — no need to have photographed every step.

1. Worker 7 dies mid-groupBy — the scheduler notices it stopped heartbeating
2. It looks up which partitions worker 7 was producing
3. It re-schedules those partitions on other workers, re-running the DAG from the last cached stage
4. The failed stage finishes; the job continues — no manual checkpoint step needed

CAVEAT: long DAGs get expensive to recompute. For deep pipelines, manually .persist() or .cache() at key stages.

Worked Example — Choosing a Partition Strategy

Pipeline	Best scheme	Why
Clickstream grouped by user_id	Hash(user_id)	Zero shuffle on that groupby/join
Sensor readings queried by date	Range(date)	Range scans stay local
Random ID lookups, no pattern	Row range	Simplicity wins, nothing to optimise for
Clickstream, 1 bot = 40% of rows	Hash + salting	Plain hash would hotspot one worker

The last row previews Session 3's 'salted groupby' pattern — partitioning and skew-handling are two sides of the same coin.

Common Mistakes & How to Spot Them

Mistake	How it shows up	Fix
Partitioning by the wrong column	Every groupby/join triggers a full shuffle	Partition by the column you group/join on most
Calling .compute() too early/often	Losing the benefit of lazy optimisation	Build the full plan, compute once at the end
Not persisting a reused expensive stage	The same costly join gets recomputed repeatedly	Add .persist()/.cache() before branching to multiple outputs
Ignoring key skew	One worker takes hours while others idle	Check for skew before choosing hash partitioning